

TP 3 : Prototypage rapide d'applications pour l'IoT

I. Introduction

L'objectif de ce TP est de finaliser l'intégration des concepts abordés précédemment en développant une application de haut niveau. Après avoir maîtrisé les protocoles de communication et le middleware oneM2M, nous utilisons Node-RED pour mettre en place les échanges entre plusieurs dispositifs. Cette approche de prototypage rapide nous permet de traiter les données issues de capteurs ou simulés via MQTT. Nous proposons une architecture complète, allant des données de capteurs jusqu'à leur visualisation et l'automatisation de commandes d'actionneurs.

II. Développement de l'application avec Node-RED

A. Test simple : Récupération et affichage des données

L'objectif de cette partie est de prendre en main Node-RED. On établit la communication entre le broker MQTT et notre application de gestion. Nous avons mis en place un flux permettant de souscrire aux différents topics utilisés pour visualiser les données reçues.

Nous avons configuré trois nœuds de type "mqtt in" connectés au broker local. Chaque nœud a été paramétré pour écouter un topic spécifique :

insa/gei/chaban/lampadaire : pour récupérer l'état de la LED.

insa/gei/chaban/capteur : pour les mesures provenant du capteur de luminosité.

insa/gei/chaban/bouton : pour récupérer l'état du bouton.

En sortie de chacun de ces nœud, nous avons utilisé un nœud "Debug" pour visualiser en temps réel les messages reçus.

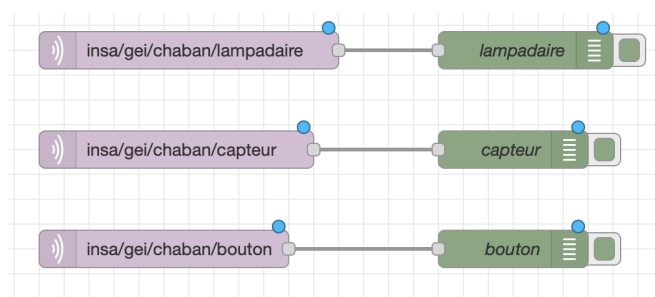


Figure 1 : Topics et debug sur NodeRED.

Cela nous a permis de vérifier que la connexion avec le broker était opérationnelle. La Figure 2 montre les messages reçus.

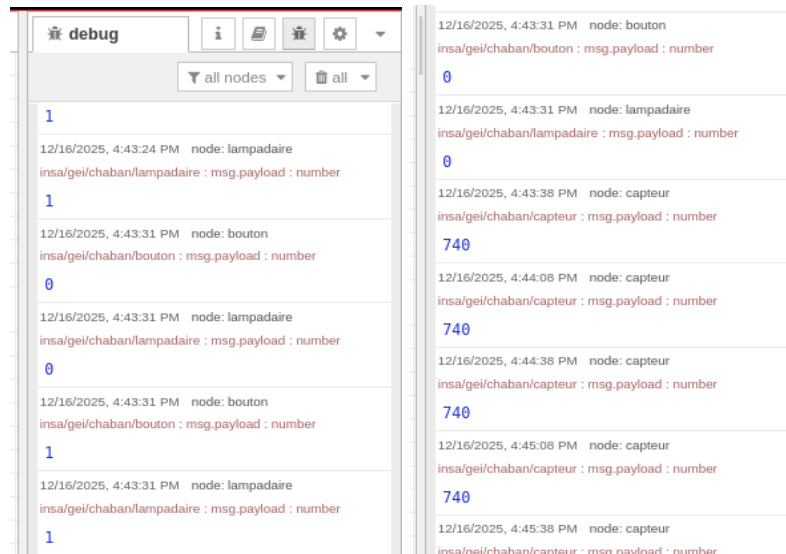


Figure 2 : Réception des messages.

B. Capteurs et actionneurs

Pour automatiser l'allumage de la LED, nous avons utilisé un nœud "Switch" pour comparer la valeur de luminosité reçue à un seuil que nous avons fixé (300).

Le nœud "Switch" reçoit la valeur numérique de la LED. Il possède deux sorties : la première est activée si la valeur est inférieure à 300, la seconde si elle est supérieure ou égale. Selon la sortie activée, un nœud vient modifier le contenu du message (msg.payload). Le message devient donc la chaîne de caractères "ON" ou "OFF", selon le cas.

Ce nouveau message est envoyé vers le topic *insa/gei/chaban/led*. L'ESP, qui est abonné à ce topic, reçoit l'ordre et pilote la LED.

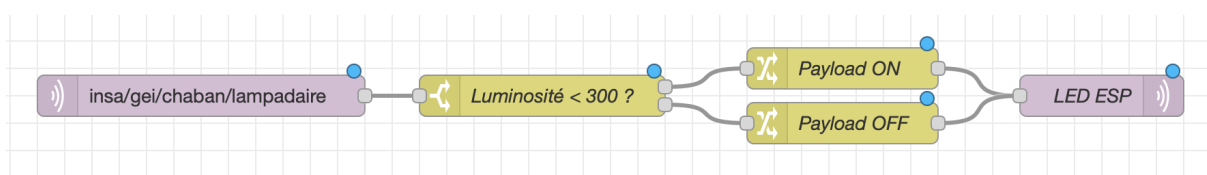


Figure 3 : Flux NodeRED permettant d'automatiser l'allumage de la LED.

C. Capteurs et actionneurs

Nous avons conçu une interface graphique (Dashboard) permettant de centraliser le monitoring et le contrôle de notre système. Le nœud Gauge permet de visualiser en temps réel la luminosité reçue sur le topic *insa/gei/chaban/lampadaire*. Le nœud "Historique" du type "Chart" permet de suivre l'évolution des données du capteur. Finalement, un nœud "Switch" permet le pilotage manuel de la LED de l'ESP. Il est relié au topic *insa/gei/chaban/led*.

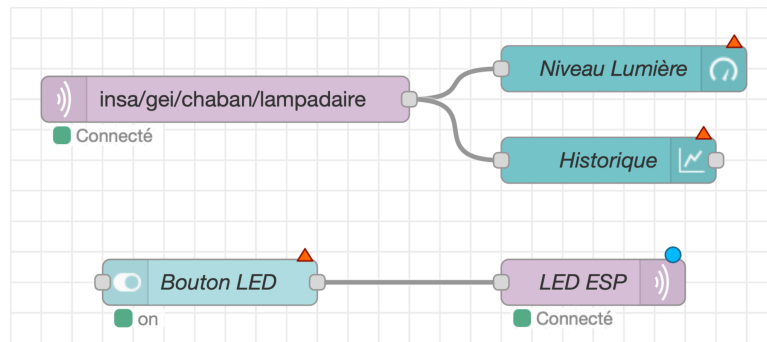


Figure 4 : Flux NodeRED pour le dashboard.

Ce tableau de bord est accessible via une interface web. Voici le résultat obtenu :

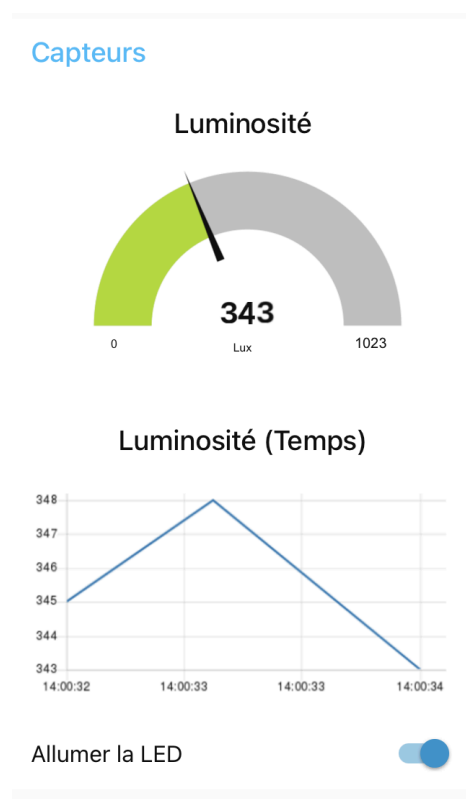


Figure 5 : Dashboard obtenu.

III. Avantages et inconvénients de NodeRED

Avantages :

Il est assez simple et rapide de prototyper des applications avec NodeRED. Passer d'une idée à une application fonctionnelle, avec interface graphique et logique se fait en quelques minutes. L'aspect visuel facilite la compréhension de l'architecture globale du flux de données

Node-RED est aussi très bon dans le mélange de protocoles. Nous avons pu faire communiquer du MQTT (ESP8266) avec du HTTP ou des services web sans écrire des codes complexes pour convertir.

Inconvénients :

Puisque toute l'intelligence est centralisée dans Node-RED, si le serveur s'arrête, l'automatisation s'interrompt totalement. Les objets deviennent "bêtes" s'ils n'ont pas de logique locale de secours.

Il est plus difficile de gérer des versions (Git) sur des fichiers JSON que sur du code source classique. Aussi, pour des milliers de nœuds, l'interface graphique peut devenir illisible.

Pour des traitements de données massifs ou des calculs mathématiques, une application compilée (en C++ ou autre) sera toujours plus performante qu'un environnement basé sur Node.js comme Node-RED.

IV. Conclusion

Ce TP a permis de valider l'ensemble de la chaîne de communication IoT par l'utilisation de Node-RED. Dans un premier temps, la connectivité entre le broker et l'application a été établie afin que les données des capteurs soient correctement réceptionnées. Par la suite, une logique d'automatisation a été implémentée pour que les actionneurs soient pilotés de manière autonome. Enfin, une interface de contrôle a été déployée, permettant une surveillance et une gestion manuelle via un dashboard web. Ce travail a démontré la rapidité du prototypage "Low-Code", bien qu'une dépendance au serveur centralisé ait été notée.